Week 14: Redis caching and choosing a database

Learning outcomes: CLO4, CLO5

Lesson

Redis keeps its data in memory and looks values up by key, which makes it very fast. It supports strings, hashes, lists, sets and sorted sets. A **TTL** (time to live) makes a key expire after a number of seconds.

In the **cache-aside** pattern, the app looks in the cache first. On a **miss**, it reads from the database, saves the result in the cache with a TTL and returns it. On a **hit**, it returns the cached value without touching the database. MySQL remains the main copy of the data. You also need a plan for when the data changes (delete or refresh the key) and for when Redis is down (read from MySQL instead).

For your own Redis structures, use province codes as keys or members, names as values and `area_km2` as the score in a sorted set. The dataset has no users or sessions, so discuss session storage on paper only.

Finally, choose the right database for each job: MySQL for related data with strict rules, MongoDB for flexible documents, Redis for fast temporary data, and a graph database when the links between items are the main question.

Worked example

Run the cache app from apps/ (`apps/README.md`) twice with the same province code and compare the two outputs. Then open `work.md`.

The cache for this week

The cache app: it stores a Redis string under the key `fese304:districts:<province_pcode>`, for example `fese304:districts:KH01`. The value is a JSON array of districts with `district_pcode` and `district_name`, read from MySQL. The key expires after 60 seconds.

For the hash, list, set and sorted-set tasks, use your own keys starting with `fese304:lab14:`. Use province codes as identifiers, names as labels and `area_km2` as the sorted-set score.

The tables and columns are described in the data guide (`data/README.md`).

Lab tasks

1. Run `apps/python/cache.py` or `apps/node/cache.mjs` twice with the same province code. Save the miss and then the hit.
2. Use redis-cli to look at the stored JSON and its TTL. Wait until the key expires and run the app again to see a miss.
3. Build a hash, a list and a set from fields in the province or district data.
4. Build a sorted set of provinces scored by area and compare its order with the same ORDER BY in MySQL. Explain what Redis does when two scores are equal.
5. Delete your cache key and check that the next run loads it again. Write a short policy for what to do when the data changes and when Redis is down.

Homework

1. Submit your Redis commands, their output, what you saw when the key expired, and your cache code.
2. For each way your capstone uses data, choose MySQL, MongoDB or Redis and explain why (durability, relationships, validation and consistency).

3. Explain why Redis is fast for key lookups, and describe one requirement where a graph database would fit better than MySQL, MongoDB or Redis.

Capstone sprint

C14 / Checkpoint 4: Deliver your Redis feature and your NoSQL model, and rehearse the final demo.

Submit

Write your answers in `work.md` and label each one, for example a heading such as `## Lab 3`. Answer the questions in `reflection.md` and save your query output in `results/`. Then commit and push to your repository. See SUBMISSIONS.md for details.